

# Giới thiệu về Polars

## **Thành viên thực hiện:**

Trần Quang Trường - 24521901

Hà Xuân Thiện - 24520031

Đinh Mạnh Hùng - 24520014

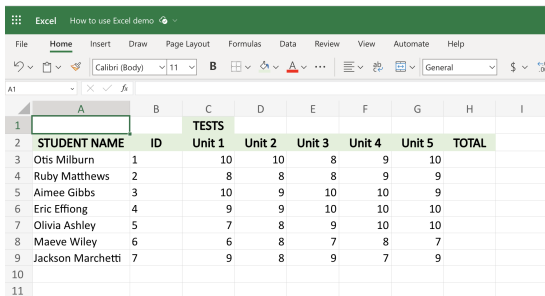
Ngày 2 tháng 2 năm 2026

## 1 So sánh Excel, Pandas và Polars

## 2 Lazy Loading và File Parquet

# Định nghĩa

**Excel** là phần mềm bảng tính (spreadsheet) dùng để lưu và quản lý dữ liệu dạng bảng, cho phép tính toán, tạo biểu đồ, phân tích dữ liệu,... Excel được phát hành lần đầu vào 30/8/1985 cho dòng máy Apple Macintosh.



The screenshot shows the Microsoft Excel interface. The title bar reads 'Excel - How to use Excel demo'. The ribbon includes 'File', 'Home', 'Insert', 'Draw', 'Page Layout', 'Formulas', 'Data', 'Review', 'View', 'Automate', and 'Help'. The 'Home' ribbon is active, showing font settings (Calibri, size 11), bold, and alignment options. The spreadsheet has a green header row (row 1) with the word 'TESTS' in cell C1. Row 2 contains the following headers: 'STUDENT NAME' in A2, 'ID' in B2, 'Unit 1' in C2, 'Unit 2' in D2, 'Unit 3' in E2, 'Unit 4' in F2, 'Unit 5' in G2, and 'TOTAL' in H2. Rows 3 through 9 contain student data. Row 10 is empty, and row 11 is partially visible.

|    | A                 | B  | C      | D      | E      | F      | G      | H     | I |
|----|-------------------|----|--------|--------|--------|--------|--------|-------|---|
| 1  |                   |    | TESTS  |        |        |        |        |       |   |
| 2  | STUDENT NAME      | ID | Unit 1 | Unit 2 | Unit 3 | Unit 4 | Unit 5 | TOTAL |   |
| 3  | Otis Milburn      | 1  | 10     | 10     | 8      | 9      | 10     |       |   |
| 4  | Ruby Matthews     | 2  | 8      | 8      | 8      | 9      | 9      |       |   |
| 5  | Aimee Gibbs       | 3  | 10     | 9      | 10     | 10     | 9      |       |   |
| 6  | Eric Effiong      | 4  | 9      | 9      | 10     | 10     | 10     |       |   |
| 7  | Olivia Ashley     | 5  | 7      | 8      | 9      | 10     | 10     |       |   |
| 8  | Maeve Wiley       | 6  | 6      | 8      | 7      | 8      | 7      |       |   |
| 9  | Jackson Marchetti | 7  | 9      | 8      | 9      | 7      | 9      |       |   |
| 10 |                   |    |        |        |        |        |        |       |   |
| 11 |                   |    |        |        |        |        |        |       |   |

# Định nghĩa

**Pandas** là thư viện Python làm việc với dữ liệu dạng bảng DataFrame, là công cụ để xử lý, phân tích dữ liệu, được sử dụng rộng rãi trong các bài toán ML hiện nay. Pandas bắt đầu phát triển năm 2008 và được mở mã nguồn năm 2009.

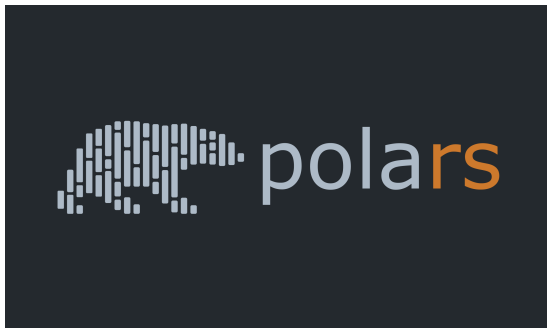
```
In [2]: 1 # Load our necessary libraries
        2 import pandas as pd
        3 import numpy as np
```

```
In [3]: 1 # Create data into CSV (that we'll import later on)
        2 # Let's say the data is for a veterinarian keeping tabs on his clients.
        3
        4 raw_data = {'pet_name': ['Woof', 'Chester', 'Rex', 'Mystery', 'Pumpkin'],
        5          'pet_last_name': ['Smith', 'Kim', '', 'Taylor', ""],
        6          'good_pet_score': [96, 34, 89, 92, 79],
        7          'type': ['dog', 'cat', 'mini-dinosaur', "unknown", "bird"],
        8          'amount_owed': ['5000', '9,000', '570, 622, 190']}
        9 df = pd.DataFrame(raw_data, columns = ['pet_name', 'pet_last_name', 'good_pet_score', 'type', 'amount_owed'])
       10 df
```

```
Out[3]:
```

|   | pet_name | pet_last_name | good_pet_score | type          | amount_owed |
|---|----------|---------------|----------------|---------------|-------------|
| 0 | Woof     | Smith         | 96             | dog           | 5000        |
| 1 | Chester  | Kim           | 34             | cat           | 9,000       |
| 2 | Rex      |               | 89             | mini-dinosaur | 570         |
| 3 | Mystery  | Taylor        | 92             | unknown       | 622         |
| 4 | Pumpkin  |               | 79             | bird          | 190         |

**Polars** là thư viện Python làm việc với dữ liệu DataFrame tương tự như pandas, được viết bằng ngôn ngữ Rust, được Ritchie Vink ra mắt lần đầu vào năm 2020. Điểm mạnh của Polars nằm ở đa luồng hiệu quả, tối ưu tốc độ chạy và tiết kiệm bộ nhớ.



# So sánh tổng quan

- **Excel**: thao tác thủ công, trực quan, phù hợp dữ liệu nhỏ và phân tích nhanh.
- **Pandas**: xử lý dữ liệu dạng bảng bằng DataFrame trong Python, có nhiều tính năng và có thể xử lý số lượng dữ liệu lớn hơn excel, thư viện phổ biến và có hệ sinh thái lớn.
- **Polars**: xử lý dữ liệu bằng DataFrame như Pandas, nhưng tối ưu hơn về thời gian chạy, bộ nhớ, vì thế cho phép xử lý trên số lượng dữ liệu lớn hơn so với Pandas. Nhưng do tuổi đời còn trẻ nên Polars chưa được biết đến rộng rãi.

- **Excel**: nhanh với thao tác thủ công trên dữ liệu nhỏ; khó mở rộng lên dữ liệu lớn.
- **Pandas**: Hiệu năng nhìn chung đáp ứng tốt đa số nhu cầu xử lý dữ liệu, nhưng khi dữ liệu rất lớn hoặc pipeline có nhiều phép biến đổi nặng thì thường bị giới hạn và khó tận dụng đa luồng một cách tự động.
- **Polars**: nhanh nhất trong ba lựa chọn khi xử lý dữ liệu, đặc biệt khi dữ liệu lớn hơn hoặc pipeline dài hơn, vì được thiết kế tối ưu hiệu năng và tốc độ.

Nguồn benchmark từ: <https://pola.rs/posts/benchmarks/>

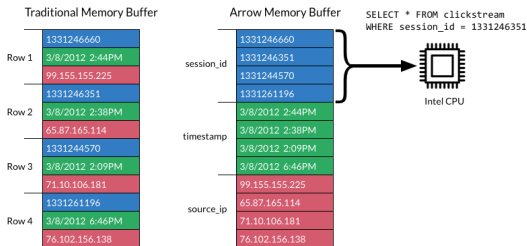




- **Excel:** tốn nhiều RAM vì ngoài dữ liệu còn có workbook, định dạng ô, công thức, pivot cache, biểu đồ, undo history,...
- **Pandas:** RAM phụ thuộc *dtype*; một số thao tác tạo bản sao làm tăng peak RAM.
- **Polars:** Thường tiết kiệm RAM hơn Pandas, dữ liệu lưu theo dạng *Apache Arrow Columnar* của Rust và có kiểu dữ liệu chặt chẽ. Cơ chế lazy đẩy filter xuống sớm và hạn chế tạo bản copy, nên peak RAM usage thường thấp hơn.

Polars lưu dữ liệu trong RAM dưới dạng Apache Arrow columnar:

|       | session_id | timestamp       | source_ip      |
|-------|------------|-----------------|----------------|
| Row 1 | 1331246660 | 3/8/2012 2:44PM | 99.155.155.225 |
| Row 2 | 1331246351 | 3/8/2012 2:38PM | 65.87.165.114  |
| Row 3 | 1331244570 | 3/8/2012 2:09PM | 71.10.106.181  |
| Row 4 | 1331261196 | 3/8/2012 6:46PM | 76.102.156.138 |



Pandas lưu dữ liệu trong RAM chủ yếu dưới dạng các mảng NumPy. Dù cũng được lưu dưới dạng cột nhưng điểm khác là ở chỗ dtype của Pandas thường là các Object dẫn tới tình trạng overhead.

- **Excel**: phổ biến trong môi trường văn phòng/doanh nghiệp; tích hợp tốt với hệ Microsoft Office.
- **Pandas**: hệ sinh thái lớn trong Python; nhiều thư viện ML / EDA mặc định hỗ trợ (scikit-learn, statsmodels, seaborn,...).
- **Polars**: hệ sinh thái đang phát triển nhanh; mạnh về Apache Arrow / Parquet và ETL hiện đại.

# Bảng so sánh

| Tiêu chí               | Excel | pandas   | Polars          |
|------------------------|-------|----------|-----------------|
| Quy mô dữ liệu phù hợp | Nhỏ   | Vừa–lớn  | Lớn             |
| Tốc độ                 | Thấp  | Tốt      | Rất tốt         |
| RAM khi dữ liệu lớn    | Cao   | Cao–TB   | TB–thấp         |
| Hệ sinh thái trong ML  | Thấp  | Rất mạnh | Đang phát triển |
| Độ dễ dùng ban đầu     | Dễ    | TB       | TB–khó          |

# DataFrame và LazyFrame

- **DataFrame**: dữ liệu đã được nạp vào bộ nhớ.
  - Khi ta gọi `filter/select/group_by`, chương trình sẽ thực thi và trả kết quả ngay.
- **LazyFrame**: các truy vấn sẽ được lưu vào *kế hoạch truy vấn (query plan)*.
  - *Kế hoạch truy vấn (Query plan)* là toàn bộ các bước ta muốn truy vấn trên dữ liệu.
  - Khi gọi `collect()` thì Polars tối ưu query plan rồi mới thực thi để tạo DataFrame kết quả.

# Eager execution và Lazy execution

- **Eager execution** làm việc với DataFrame: các hàm `read_*` trả về DataFrame, và các thao tác như `filter/select/group_by` chạy ngay rồi trả ra kết quả là DataFrame.
- **Lazy execution** làm việc với LazyFrame, các hàm `scan_*` trả về LazyFrame và các thao tác chỉ được ghi vào kế hoạch truy vấn; đến khi gọi `collect()` thì mới chạy và trả về DataFrame kết quả.

# Parquet là gì?

## Row-oriented dataset in user interface

| ID  | Name | Age |
|-----|------|-----|
| 111 | John | 12  |
| 112 | Jane | 9   |

## Row-oriented dataset saved on disk

|     |      |    |     |      |   |
|-----|------|----|-----|------|---|
| 111 | John | 12 | 112 | Jane | 9 |
|-----|------|----|-----|------|---|



# Parquet là gì?

## Columnar database in user interface

| ID  | Name | Age |
|-----|------|-----|
| 111 | John | 12  |
| 112 | Jane | 9   |

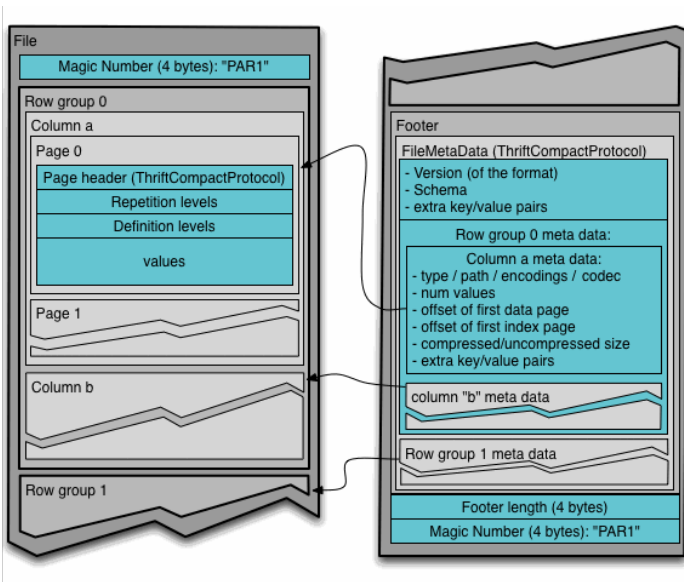
## Columnar dataset stored on disk

111 112 John Jane 12 9

# Parquet là gì?

- **Parquet** là định dạng lưu trữ *dữ liệu dạng cột(columnar)*.
- Trong **Parquet**, dữ liệu được lưu thành nhiều row group.

# Parquet là gì?



# Parquet là gì?

- Parquet chia dữ liệu thành các row groups để hỗ trợ đọc theo từng phần thay vì phải đọc cả file một lần. Bên trong mỗi row group, dữ liệu vẫn được tổ chức theo cột, nên vừa đọc chọn lọc cột được vừa có thể skip theo khối.
- Parquet lưu metadata và thống kê trong mỗi row group, giúp engine quyết định có cần đọc khối đó không. Nhờ vậy, các điều kiện lọc đơn giản có thể giúp skip qua nhiều row, tiết kiệm chi phí tính toán.
- Schema trong Parquet được dùng để lưu tên cột và kiểu dữ liệu của từng cột. Nhờ schema nên Polars không phải đoán kiểu dữ liệu như CSV khi đọc, nên sẽ đọc nhanh và ổn định hơn.

# Quá trình lazy loading trong Polars

- Lazy loading gồm 3 bước: **Build plan** → **Optimize** → **Execute**.

# Bước 1: Build plan

- **Build Plan** là giai đoạn Polars thu thập và ghép các transformation thành một kế hoạch truy vấn.
- **Query plan** là kế hoạch xử lý dữ liệu mà Polars tạo ra qua lazy API, nó ghi lại toàn bộ các bước theo đúng pipeline ta viết.

## Bước 2: Optimize

- **Optimize** là giai đoạn Polars biến plan ban đầu thành plan tốt hơn bằng cách rút gọn, sắp xếp lại và đẩy các bước xuống gần nguồn đọc dữ liệu.

## Bước 3: Execute

- **Execute** là giai đoạn engine chạy plan đã tối ưu từ bước 2 và tạo ra kết quả.



# Các tối ưu quan trọng trong lazy

- Các tối ưu trong lazy loading có chung mục tiêu là giảm lượng dữ liệu càng sớm càng tốt trước khi vào các phép tính nặng (`group_by/join/sort`).

# Các tối ưu quan trọng trong lazy

- **Projection pushdown:** là tối ưu chỉ đọc các cột cần cho kết quả cuối, thay vì đọc toàn bộ cột rồi mới bỏ bớt.
- **Predicate pushdown:** là tối ưu lọc càng sớm càng tốt, tức là đẩy điều kiện filter xuống ngay lúc đọc dữ liệu thay vì đọc hết rồi mới lọc.
- **Pruning:** là tối ưu cắt bỏ phần không cần trong kế hoạch truy vấn, như các cột hoặc bước trung gian không còn được dùng ở kết quả cuối.

# Vì sao Parquet hợp với lazy loading?

- **Projection pushdown + columnar:**

Vì Parquet được lưu theo dạng cột, projection pushdown cho phép Polars chỉ đọc đúng các cột cần thiết thay vì đọc toàn bộ.

- **Predicate pushdown + metadata:**

Với metadata của Parquet, predicate pushdown có thể giúp bỏ qua các row group không thể thỏa điều kiện từ ban đầu.

⇒ Khi projection pushdown giảm số cột và predicate pushdown giảm số dòng từ ngay khâu đọc, toàn bộ pipeline phía sau sẽ nhẹ đi đáng kể. Từ đó, giúp group\_by/join/sort chạy nhanh hơn, dùng ít RAM hơn, và thời gian tổng thể giảm đi đáng kể.